Getting Torque, RPM & Power to Read

Flash-ready fix for Conv_Simple_1.9 (no variable changes)

The short version

Your GS10 drive is perfect - the keypad shows real torque (73.7%) and speed (880 rpm at 30 Hz). The real problem turned out to be simpler than we thought: the program currently RUNNING on the PLC is the older V1.8 version, which only reads the BASIC drive values (frequency, current, DC bus, fault). It never reads torque, rpm or power at all - so they sit at 0.

The corrected program is built and ready. It adds the torque/rpm/power read. It needs NO variable changes - just flash it.

Why it was 0 (in plain terms)

- The drive keeps its live numbers in a list of 'registers'.
- The flashed V1.8 program only ever asked for the first batch (basic readings).
- Torque, rpm and power live further down that list - and were simply never requested.

So this was never a wiring or drive-setting problem (you proved that on the keypad). The PLC program just wasn't reading those three registers.

The fix (in plain terms)

The new program reads the drive in TWO small batches, taking turns each cycle: one cycle it grabs the basic readings, the next cycle it grabs torque + rpm + power. Both batches are small enough to fit the memory the program ALREADY has, so nothing in the variable list has to change. It is still one message per cycle on the RS-485 wire.

How to flash it

1. Open the Conv_Simple_1.9 project in Connected Components Workbench. The corrected program is already saved into the project file, so opening it loads the new version.
2. Build, then Download to the PLC. (No variable changes, no Modbus-map changes.)
3. That's it - tell Claude to re-run the live logger. Torque, rpm and power should now read real values that track the keypad (about 73% torque, ~880 rpm at 30 Hz).

If, after Download, torque/rpm/power are STILL 0 AND the basic readings still work, then CCW opened a cached old copy of the program instead of the new one. In that case do the paste fallback below.

Paste fallback (only if step 2 loaded the old program)

1. In CCW open the Prog_init program, select ALL of it, and delete it.
2. Open the file plc\Prog_init_ConvSimple_v1.9.st (in the MIRA repo), copy the whole thing, and paste it in as the new Prog_init.
3. Build, then Download. (Still no variable changes.)

What changed (the read section)

For reference, this is the new read logic - it replaces the old single-batch read. The 'taking turns' is the lp_toggle line; torque/rpm/power are read on the TRUE turn, the basic values on the FALSE turn. Everything fits read_data[1..10] as-is.

```
(* --- 500 ms tick; alternate read/write, and alternate the two read blocks --- *)
poll_timer(IN := NOT poll_timer.Q, PT := T#500ms);

IF poll_timer.Q THEN
    vfd_poll_count := vfd_poll_count + 1;
    poll_phase := NOT poll_phase;          (* read tick <-> write tick *)
    IF poll_phase THEN
        lp_toggle := NOT lp_toggle;        (* on READ ticks: swap the two blocks *)
    END_IF;
END_IF;

(* lp_toggle FALSE -> monitor 0x2100..0x2106 ;  TRUE -> load/power 0x210B..0x210F *)
read_local_cfg.Channel     := 2;
read_local_cfg.TriggerType := 0;
read_local_cfg.Cmd         := 3;            (* FC03 read holding *)
read_target_cfg.Node       := 1;

IF lp_toggle THEN
    read_local_cfg.ElementCnt := 5;         (* 0x210B..0x210F *)
    read_target_cfg.Addr      := 16#210C;   (* Addr=wire+1 -> wire 0x210B *)
ELSE
    read_local_cfg.ElementCnt := 7;         (* 0x2100..0x2106 *)
    read_target_cfg.Addr      := 16#2101;   (* Addr=wire+1 -> wire 0x2100 *)
END_IF;

mb_read_status(IN        := poll_timer.Q AND poll_phase,
               LocalCfg  := read_local_cfg,
               TargetCfg := read_target_cfg,
               LocalAddr := read_data);

IF mb_read_status.Q AND NOT mb_read_status.Error THEN
    IF lp_toggle THEN
        vfd_torque    := read_data[1];      (* 0x210B torque % x10 *)
        vfd_motor_rpm := read_data[2];      (* 0x210C motor rpm    *)
        vfd_power     := read_data[5];      (* 0x210F power kW x1000 *)
    ELSE
        vfd_fault_code  := read_data[1];    (* 0x2100 fault       *)
        vfd_status_word := read_data[2];    (* 0x2101 status      *)
        vfd_frequency   := read_data[4];    (* 0x2103 Hz x100     *)
        vfd_current     := read_data[5];    (* 0x2104 A x100      *)
        vfd_dc_bus      := read_data[6];    (* 0x2105 V x10       *)
        vfd_voltage     := read_data[7];    (* 0x2106 V x10       *)
        IF vfd_fault_code > 0 THEN
            vfd_last_fault := vfd_fault_code;
        END_IF;
    END_IF;
    vfd_comm_ok := TRUE;
END_IF;
IF mb_read_status.Error THEN
    vfd_comm_ok   := FALSE;
    vfd_err_count := vfd_err_count + 1;
END_IF;
```

How to know it worked

Success:

The logger shows torque/rpm/power moving with the motor and matching the keypad. rpm is about (Hz x 120 / poles) minus a little slip - e.g. ~880 rpm at 30 Hz on a 4-pole motor. The basic readings (Hz, amps, DC bus) keep working - they now just refresh every other cycle, which is still about twice a second.

If they are STILL 0:

Then the drive is not answering a read of registers 0x210B..0x210F over RS-485 (an addressing detail, not this code). Tell Claude - that would be the next thing to check. But since the keypad shows the values, this fix should do it.

What this does NOT change

- No variable-table edits, no Dimension changes (this is why we read in two halves).
- The motor control (forward / reverse / stop) - untouched.
- The Modbus map / the drive's own settings - untouched.